

Defansif Zararlı Yazılım Tarama Sisteminin C++ ve Nesne Yönelimli Programlama ile Tasarımı

Emirhan

Bilgisayar Mühendisliği Bölümü

Kastamonu Üniversitesi

Kastamonu, Türkiye

[E-posta Adresin]

Özet— Bu makale, zararlı yazılımların tespitine yönelik nesne yönelimli programlama (OOP) prensipleri kullanılarak C++ dilinde geliştirilen defansif bir tarama sisteminin tasarımını ve uygulamasını sunmaktadır. Geliştirilen sistem, modüler bir mimari üzerine inşa edilmiş olup, çok biçimlilik (polymorphism) ve kapsülleme (encapsulation) tekniklerinden faydalanmaktadır. Sistem içerisinde, bilinen zararlı yazılım kalıplarını kontrol eden imza tabanlı (signature-based) bir motor ve dosya içerisindeki şüpheli komut dizilerini analiz eden sezgisel (heuristic) bir motor olmak üzere çift katmanlı bir analiz mekanizması bulunmaktadır. Yapılan testler, sistemin zararlı dosyaları başarıyla tespit ettiğini ve analiz sonuçlarını kalıcı bir kayıt (log) dosyasına raporladığını göstermiştir.

Anahtar Kelimeler— zararlı yazılım analizi, nesne yönelimli programlama, antivirüs, C++, siber güvenlik

I. Giriş

Günümüzde siber tehditlerin karmaşıklığı ve sayısındaki artış, sistem güvenliğini sağlamak adına defansif yazılım geliştirme pratiklerini zorunlu kılmaktadır. Zararlı yazılım analizi (malware analysis), uç nokta (endpoint) sistemlerini korumanın en temel adımlarından biridir. Bu bağlamda, dosyaların içeriklerini ve davranışsal özelliklerini inceleyerek olası tehditleri tespit edebilen tarama motorları, antivirüs sistemlerinin çekirdeğini oluşturmaktadır.

Bu projenin temel amacı, C++ programlama dilinin sunduğu düşük seviyeli bellek yönetimi avantajlarını ve Nesne Yönelimli Programlama (OOP) mimarisinin sağladığı modülerliği bir araya getirerek, genişletilebilir bir defansif dosya tarama sistemi geliştirmektir. Yazılım, sadece bilinen tehdit veri tabanlarını kontrol eden geleneksel yöntemlerle sınırlı kalmayıp, şüpheli komut çağrılarını da analiz edebilen çift katmanlı bir yapıda tasarlanmıştır.

II. Sistem Mimarisi ve Metodoloji

Geliştirilen tarama sistemi, Tek Sorumluluk Prensibi'ne (Single Responsibility Principle) sadık

kalınarak birbirleriyle entegre çalışan modüller sınıflara (class) ayrılmıştır. Mimari; dosya yönetim katmanı, polimorfik analiz motorları ve merkezi raporlama biriminden oluşmaktadır.

A. Dosya Yönetimi ve Kapsülleme

Sistemin girdi mekanizması `FileAnalyzer` sınıfı tarafından yönetilmektedir. Bu sınıf, hedef dosyaların bellek sızıntısına yol açmadan güvenli bir şekilde ikili (binary) formatta okunmasından sorumludur. Sınıf içerisindeki dosya yolları ve içerik verileri, dışarıdan doğrudan erişime kapatılarak kapsülleme (encapsulation) prensibi ile güvence altına alınmış ve sadece kontrollü metotlar aracılığıyla erişime açılmıştır.

B. Çok Biçimlilik ve Tarama Motorları

Analiz motorlarının tasarımı, sistemin ileride yeni tarama tekniklerine açık olması amacıyla `IScanner` adlı bir soyut arayüz (interface) üzerine inşa edilmiştir. Bu arayüz üzerinden çok biçimlilik (polymorphism) yeteneği kullanılarak iki farklı güvenlik katmanı oluşturulmuştur:

- *Sezgisel Tarayıcı (Heuristic Scanner)*: Dosya içeriğinde yer alan ve zararlı potansiyeli taşıyan şüpheli sistem komutlarını, betik dizilerini veya hafıza manipülasyon (örneğin; system(), eval()) çağrılarını dinamik olarak analiz eder.
- *İmza Tabanlı Tarayıcı (Signature Scanner)*: `SignatureDatabase` sınıfı ile entegre çalışarak, taranan dosya içeriklerini ve özetlerini bilinen zararlı yazılım imzalarıyla (signature matching) milisaniyeler içerisinde eşleştirerek kesin tehditleri yakalar.

C. Merkezi Analiz Motoru ve Raporlama

Sistemin tüm tarayıcı modüllerin yönetimi ve dosya yönlendirmeleri `AntivirusEngine` adlı merkezi motor tarafından sağlanmaktadır. Birleştirme (composition) yöntemiyle tarayıcıları bünyesinde barındıran bu motor, analiz süreci tamamlandığında `ReportGenerator` sınıfını tetikleyerek işlem sonuçlarını, tespit edilen tehditleri ve zaman damgasını (timestamp) kalıcı bir rapor dosyasına (log) kaydetmektedir.

III. Test ve Bulgular

Geliştirilen sistemin doğruluğunu, tespit yeteneğini ve hata payını ölçmek amacıyla, farklı özelliklere sahip üç adet test dosyası kullanılarak kontrollü bir analiz simülasyonu gerçekleştirilmiştir. Testler; tamamen temiz bir metin dosyası, şüpheli işletim sistemi komutları içeren bir betik (script) dosyası ve bilinen bir zararlı yazılım imzası barındıran sahte bir virüs dosyası üzerinde yürütülmüştür.

- *Temiz Dosya Analizi*: İçerisinde yalnızca standart metin verisi bulunan dosya sisteme yüklendiğinde, her iki tarama motoru da dosyayı incelemiş ve herhangi bir yanlış pozitif (false-positive) uyarı üretmeden dosyayı "Güvenli" olarak işaretlemiştir.
- *Sezgisel Tespit*: İçerisinde system() gibi işletim sistemi seviyesinde çağrılar barındıran şüpheli bir .bat dosyası analiz edildiğinde, Sezgisel Motor (Heuristic Scanner) imza veri tabanına ihtiyaç duymadan dosyadaki anormal davranış potansiyelini tespit etmiş ve işlemi durdurmuştur.
- *İmza Tabanlı Tespit*: Veritabanına önceden tanımlanmış bir zararlı hash/imza değerini

barındıran dosya taranmış ve İmza Tabanlı Motor (Signature Scanner) tarafından kesin tehdit olarak anında engellenmiştir.

Yapılan testler sonucunda, merkezi `AntivirusEngine` motorunun tüm süreçleri başarıyla koordine ettiği ve elde edilen sonuçları zaman damgası (timestamp), taranan dosya yolu ve tespit eden motor bilgisiyle birlikte `scan\_report.txt` log dosyasına eksiksiz biçimde kaydettiği gözlemlenmiştir.

IV. Sonuç

Bu çalışmada, uç nokta güvenliğini (endpoint security) sağlamaya yönelik defansif bir zararlı yazılım tarama sistemi prototipi başarıyla tasarlanmış ve C++ dili ile geliştirilmiştir. Nesne Yönelimli Programlama (OOP) prensiplerinin (kapsülleme, kalıtım, çok biçimlilik) projenin temel mimarisine entegre edilmesi, sisteme yüksek düzeyde modülerlik ve sürdürülebilirlik kazandırmıştır.

Tarama motorlarının polimorfik bir arayüz (interface) üzerinden yönetilmesi, yazılımın gelecekte makine öğrenmesi (machine learning) tabanlı davranışsal analiz gibi daha gelişmiş motorları bünyesine katabilmesine olanak tanımaktadır. Sonuç olarak geliştirilen bu sistem, modern bir antivirüs yazılımının temel çalışma prensiplerini ve yazılım mühendisliği standartlarını akademik bir çerçevede ortaya koymuştur.

Kaynaklar

1. B. Stroustrup, *The C++ Programming Language*, 4th ed., Boston, MA, USA: Addison-Wesley, 2013.
2. M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*, San Francisco, CA, USA: No Starch Press, 2012.
3. E. Gandotra, D. Bansal, and S. Sofat, "Malware Analysis and Classification: A Survey," *Journal of Information Security*, vol. 5, pp. 56-64, 2014.
4. R. C. Seacord, *Secure Coding in C and C++*, 2nd ed., Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2013.